Monte Carlo Methods for Solving Large Games

Counterfactual Regret Minimization as a Case Study

Seongwoo Hur

Department of Statistics, University of Chicago

1 Monte Carlo Methods for Solving Large Games

Many decision problems can be drawn as trees. At the root there is an initial state, then an action, then a response, then another action, and so on until the process ends. This picture is useful, but it also exposes the main computational problem: the tree can become enormous. A small poker game already has thousands of possible histories, and realistic games have far too many histories to enumerate.

Monte Carlo ideas help with that problem by turning exact tree sweeps into sampled tree walks. The exact baseline, Counterfactual Regret Minimization (CFR), learns by repeatedly walking through the whole game tree. Monte Carlo CFR (MCCFR) keeps the same learning idea, but replaces the full walk through the tree with random sampled walks. Sampling is not "free": it saves work per update, but introduces noise. The experiment below makes that tradeoff visible.

1.1 The Computational Problem

A two-player zero-sum game has one player's gain equal to the other player's loss. Rock-paper-scissors is the smallest example. Poker is a more interesting example because players move over time, and each player has some private information. In a poker hand, a player knows their own card but not the opponent's card.

The game can be represented as a tree with three kinds of points:

- chance points, where cards are dealt or a public card is revealed;
- decision points, where a player chooses an action;
- terminal points, where the payoff is known.

Hidden information adds one more idea. Several hidden histories may be indistinguishable to the player whose turn it is. Such a set of indistinguishable histories is an *information set*. A strategy says what probability the player assigns to each legal action at each information set.

The goal is to find a strategy that is hard to exploit. The experiments measure this by *exploitability*. Informally, exploitability asks: if a perfect opponent studied a strategy, how much could they gain by best-responding to it? Lower exploitability means the strategy is closer to equilibrium.

1.2 Why Exact Game Trees Become Too Large

The difficulty is not that any single decision is complicated. The difficulty is multiplication. If a player has three possible actions now, and the opponent has three possible responses, and this continues for only ten decision points, the number of possible action histories is already on the order of 3^{10}. Card deals and public chance events multiply the tree further.

This is why poker-like games are a natural place to see the value of sampling. Even a simplified game contains many possible private cards, public cards, and betting histories. Most of those histories are not visited in any one real play of the game, but full-tree methods still have to account for all of them when they compute an exact update.

Monte Carlo methods change the question. Instead of asking for the exact average over every possible hidden card and every possible future action, they ask for random evidence from some of those possibilities. A single sample is not enough, but many cheap samples can reveal the same broad direction as a much more expensive exact calculation.

1.3 Learning by Local Regret

CFR can sound technical, but the basic learning rule is simple. At every information set, keep a score for each action. The score asks: looking back over training, how much would this player regret not choosing this action more often? Actions with positive regret get more probability in the next round. Actions with no positive regret get little or no probability.

If $R(a)$ is the cumulative regret of action a, regret matching chooses

$$\sigma(a) = \begin{cases} \dfrac{[R(a)]_+}{\sum_{b \in A} [R(b)]_+}, & \text{if some action has positive regret,} \\ \dfrac{1}{|A|}, & \text{otherwise.} \end{cases}$$

The second line is the initial fallback: if all regrets are zero or negative, play uniformly at random.

CFR applies this small rule at every information set in the game. One iteration of full-tree CFR does the following.

1. Walk through every possible branch of the game tree.
2. At each information set, compare the current mixed action with the available alternatives.
3. Add regret to actions that would have done better.
4. Average the strategies produced over time.

The averaging step matters because the average strategy is the object that converges in the usual CFR theory [4]. The important computational fact is that full-tree CFR computes exact averages over the whole tree. This is clean and stable, but expensive. If the tree doubles in size, one iteration roughly doubles in cost.

1.4 Where Monte Carlo Enters

Monte Carlo methods are useful when an exact average is too expensive but random samples are cheap enough. For example, instead of computing $\mathbb{E}[f(X)]$ exactly, we can sample $X_1, \ldots, X_N$ and use

$$\frac{1}{N} \sum_{n=1}^{N} f(X_n).$$

The estimate is noisy, but it often becomes useful long before the exact calculation would be possible.

MCCFR uses the same idea inside game solving [5]. Instead of walking through the entire game tree on every update, it samples one part of the tree and uses that sample to update regrets. A sampled path from the start of the game to the end is called a trajectory. The algorithm then asks: how can one or a few sampled trajectories give useful information about the much larger tree?

Method	What one update visits	Main tradeoff
Full-tree CFR	the whole tree	stable but expensive
External sampling	sampled chance and opponent actions	cheaper, still compares player actions
Outcome sampling	one full sampled trajectory	cheapest, but highest variance

The table summarizes the core computational choice. The algorithms differ in how much of the tree they inspect before making a regret update.

1.5 A Small Sampling Picture

A small example makes the sampling idea more concrete. Imagine a player faces one information set and can choose either action A or action B. Behind that information set there may be many hidden worlds: the opponent may hold different cards, chance may reveal different public cards later, and the opponent may respond in different ways.

Full-tree CFR asks the expensive question: what would happen to action A and action B after averaging over all of those hidden worlds? That is the cleanest answer, but it requires looking everywhere.

External sampling asks a cheaper question: in one randomly chosen hidden world, how do action A and action B compare? This is not the exact average over all worlds, but it is still a useful comparison. After many random worlds, the accumulated regrets begin to point in the right direction.

Outcome sampling asks an even cheaper question: in one randomly chosen hidden world, after choosing one sampled action, what happened? Since the other action was not tried, the algorithm has to use an importance weight. That is why outcome sampling is the closest to ordinary Monte Carlo simulation, and also why it is the noisiest method here.

This example also explains why the algorithms are useful for larger games. The exact average becomes impossible first. Random local evidence can still be collected.

1.6 External Sampling

External sampling is the easier Monte Carlo version to understand. Suppose we are updating player 0. The algorithm samples the random card deal and samples player 1's actions. But when it reaches an information set for player 0, it still checks all of player 0's legal actions.

This means player 0 can still make a direct comparison such as: in this sampled world, betting led to this value, checking led to that value, so which action deserved more probability? The update is noisy because it saw only one sampled card deal and

one sampled sequence of opponent actions. But it is less noisy than using only one action of the player being trained.

External sampling can be summarized as: compare all actions of the player being trained inside one sampled world. That is why it stays close to full-tree CFR while visiting many fewer nodes.

1.7 Outcome Sampling

Outcome sampling goes further. It samples chance, the opponent's actions, and the update player's own actions. One update follows one complete play of the game. This is attractive because it is extremely cheap per update. The difficulty is that most actions were not tried, so the algorithm must correct for what it did and did not sample.

This is where importance weighting enters. If an action was sampled with probability $q(a)$ and produced value $u(a)$, the sampled value is scaled as

$$\widehat{u}(a) = \frac{u(a)}{q(a)}.$$

Rare sampled events receive larger weight. This is the same correction principle used in ordinary importance sampling: if a sample was unlikely under the sampling rule, it represents more unseen cases. The cost is variance. A few rare high-weight samples can move the estimate a lot.

So outcome sampling gives the sharpest Monte Carlo tradeoff. It touches the least of the tree, but its learning curve is much noisier.

1.8 Implementation

The algorithms are implemented in Python using a deliberately direct, tabular representation. The source code is split into small files: `cfr.py` for full-tree CFR, `external_sampling_mccfr.py` for external sampling, `outcome_sampling_mccfr.py` for outcome sampling, and `util.py` for evaluation helpers. The local learning rule is in `regret_matching.py`.

For reproducibility, the complete implementation used for the experiments, including the game definitions, trainers, evaluation utilities, and notebooks, is available in the project repository: https://github.com/swhur1214/mccfr.

The experiments use two small games. Kuhn Poker is tiny and is useful for debugging [8, 2]. Leduc Poker is still a toy game, but it is large enough to show why sampling matters [3]. The version used here has a private card, a public card, and two betting rounds. The generated tree has 9,451 nodes and 288 information sets. That is small compared with real poker, but large enough that full traversal is noticeably more expensive.

All trainers expose the same basic interface:

`train(iterations)`, `current_strategy()`, `average_strategy()`.

This made it possible to run the same experiment for full-tree CFR, external sampling, and outcome sampling. The notebooks reproduce the experiments and save the figures used below.

1.9 Sanity Checks

Before moving to Leduc Poker, the algorithms are run on Kuhn Poker. Kuhn Poker is small enough to inspect directly, and its equilibrium value is known: player 0 receives about $-1/18$ per hand under equilibrium play. As training proceeds, full-tree CFR should move toward this value, and the sampled variants should show the same direction with more noise and fewer visited nodes.

Kuhn Poker therefore gives a compact reference point. Wrong payoff signs, invalid chance probabilities, or misplaced regret updates usually appear as values that drift away from $-1/18$ instead of toward it.

1.10 Experiment

The experiment compares the three methods on Leduc Poker. Full-tree CFR runs for 500 iterations, external sampling for 100,000 iterations, and outcome sampling for 300,000 iterations. The sampled methods are averaged over three random seeds.

Algorithm	Iterations	Value	Exploitability	Visited nodes
CFR	500	-0.091170	0.030586	9.45×10^6
ES-MCCFR	100,000	-0.088230	0.041565	4.23×10^6
OS-MCCFR	300,000	-0.093453	0.237494	4.13×10^6

The value column is the expected payoff for player 0. The more important column here is exploitability: lower is better. All three methods improve over an untrained random strategy. Full-tree CFR is the smoothest because every update uses exact tree averages. External sampling gets close while visiting much less of the tree. Outcome sampling is much noisier in this simple implementation.

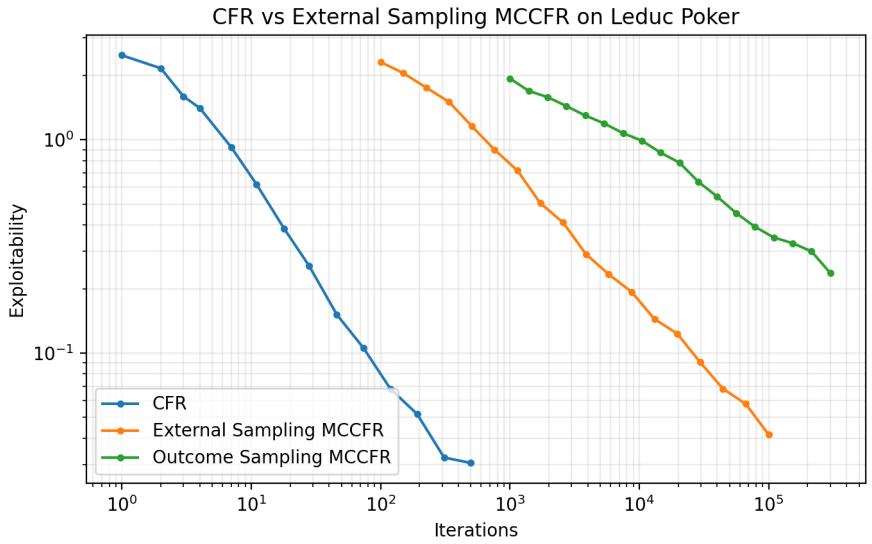

Figure 1 Exploitability on Leduc Poker as training progresses. Lower is better.

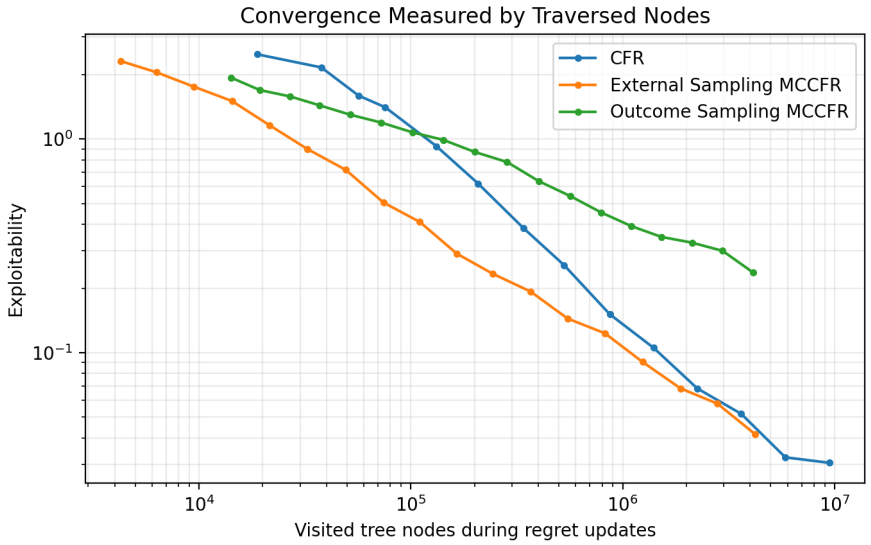

Figure 2 The same learning curves measured by how many tree nodes were visited.

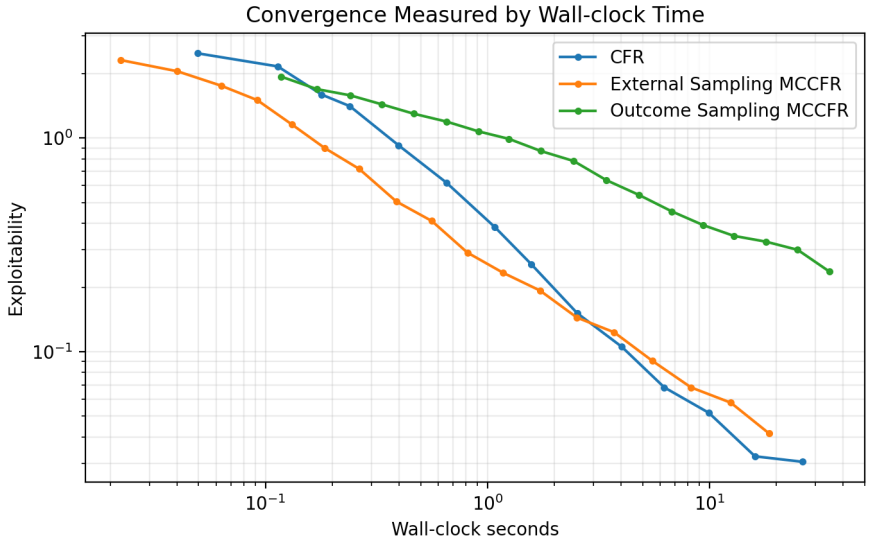

Figure 3 The same learning curves measured by elapsed wall-clock time.

The node count is the clearest way to see the Monte Carlo idea. In this implementation, full-tree CFR visits about 18,902 tree nodes per iteration. External sampling visits about 42.6 nodes per iteration, and outcome sampling visits about 13.7 nodes per iteration. The sampled methods therefore do much less work per update. They pay for this with variance and with the need for many more iterations.

The three plots put the same learning process on three different axes. Iteration count favors full-tree CFR, because one full traversal is a much stronger update than one sampled trajectory. Node count asks a more Monte Carlo question: how good is the strategy after spending a given amount of tree traversal work? Wall-clock time adds the practical engineering view. In this small pure Python run, the sampled methods are not automatically faster in elapsed seconds, because the game is still small and sampling has overhead. So the iteration plot should not be read by itself. For MCCFR, the more relevant question is what happens after the same amount of tree traversal or runtime.

1.11 What the Experiment Shows

The Leduc experiment shows the tradeoff more clearly than Kuhn Poker. Kuhn is so small that every method can touch enough of the tree quickly. In Leduc, a full CFR update already requires a much larger sweep, so the sampled methods start to make sense even though their curves are noisier.

There are three takeaways.

- Full-tree CFR is the clean baseline: it is stable because it computes exact updates, but its cost scales with the size of the tree.
- External sampling is a conservative Monte Carlo step: it samples the outside world, but still compares all actions of the player being trained. This made it the most reliable sampled method in these runs.
- Outcome sampling is the aggressive Monte Carlo step: it samples one whole trajectory and uses importance weights. It is cheap per update, but the estimates have high variance.

For this implementation, the node-count plots are the most useful ones. They show the practical reason for using MCCFR: it keeps the updates small when a full tree traversal is becoming expensive. The wall-clock plots are less dramatic here because the games are still toy-sized and the code is plain Python, but the cost difference in tree visits is already visible.

1.12 Discussion and Bibliography

MCCFR is a useful example of Monte Carlo thinking in a sequential decision problem. The original CFR algorithm asks for an exact sweep over all possible futures. MCCFR asks how much of that sweep can be replaced by random sampled futures while still producing useful learning updates.

For larger games, sampling alone is not the end of the story. Practical game-solving systems also use abstraction, better averaging, variance reduction, and careful engineering. But the core insight already appears in this small experiment: if the full

tree is too expensive, sampled trajectories can still guide the strategy toward lower exploitability.

The experiment also clarifies a limitation. Monte Carlo methods reduce one kind of cost, but they introduce another kind of difficulty: statistical noise. External sampling and outcome sampling make different choices about where to put that noise. Understanding that design choice is the main conceptual goal of the implementation.

Natural next steps are larger Leduc variants, where full-tree CFR becomes less practical, and simple variance-reduction ideas for outcome sampling. The outcome-sampling experiment makes the lesson especially clear: sampling a single trajectory is computationally attractive, but the raw estimator can be unstable. A better estimator could keep the cheap updates while making the learning curve less erratic.

References

[1] Seongwoo Hur. Study and implementation of regret matching and counterfactual regret minimization algorithms. GitHub repository, 2026. https://github.com/swhur1214/cfr.

[2] *Kuhn poker*. Wikipedia. https://en.wikipedia.org/wiki/Kuhn_poker. Accessed March 7, 2026.

[3] Finnegan Southey, Michael P. Bowling, Bryce Larson, Carmelo Piccione, Neil Burch, Darse Billings, and Chris Rayner. Bayes' bluff: Opponent modelling in poker. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, 2005. https://arxiv.org/abs/1207.1411.

[4] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems 20*, 2007. https://papers.nips.cc/paper/3306-regret-minimization-in-games-with-incomplete-information.

[5] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte Carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems 22*, 2009. https://papers.nips.cc/paper/3713-monte-carlo-sampling-for-regret-minimization-in-extensive-games.

[6] Gabriele Farina, Christian Kroer, and Tuomas Sandholm. Regret circuits: Composability of regret minimizers. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[7] Gabriele Farina. *Computational Game Solving (CMU 15-888), Lectures 1–5*. 2021.

[8] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*, 1950.